

Redes Neurais Artificiais

(Artificial Neural Network)

Edimilson B. dos Santos

Introdução

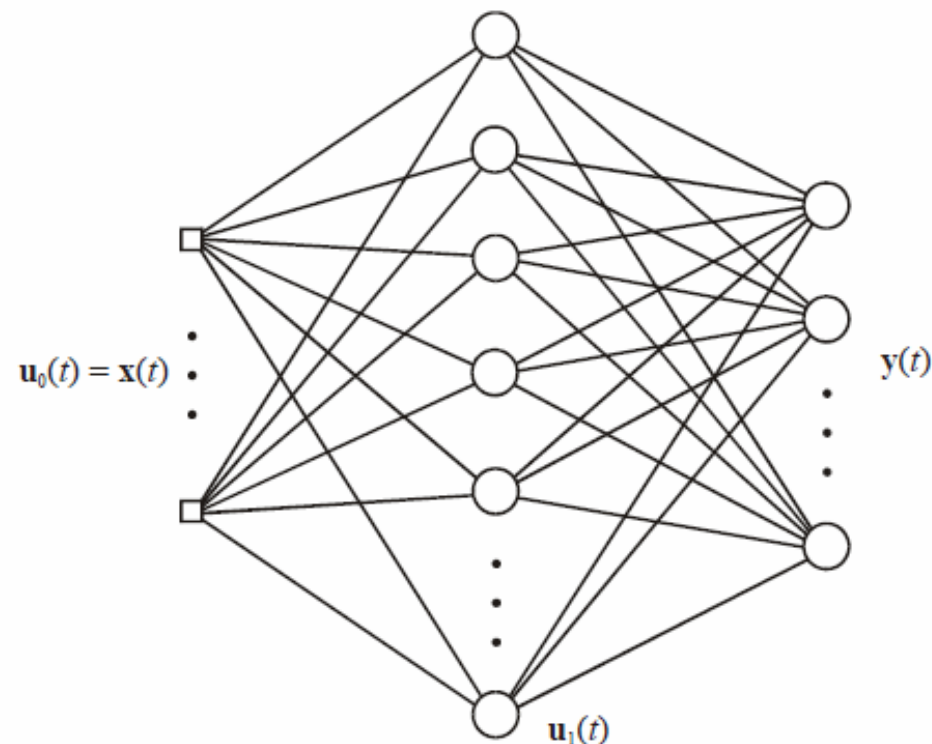
- Os modelos para regressão linear compreendem combinações lineares de funções de base fixa (não-lineares).
- Estes modelos tem propriedades computacionais e analíticas úteis, mas sua aplicação prática é limitada pela dimensionalidade.
- Para aplicar tais modelos a problemas de grande escala, é necessário ajustar as funções de base aos dados.
- Uma abordagem alternativa é fixar o número de funções base, mas permiti-las serem ajustáveis.
 - Usar formas paramétricas para funções base nas quais os valores dos parâmetros são ajustados durante o treinamento.
- O modelo mais bem sucedido deste tipo no contexto de reconhecimento de padrões é o de **Redes Neurais**.

Introdução

- Alguns dos trabalhos mais antigos de IA tiveram o objetivo de criar **Redes Neurais Artificiais (RNA)**.
 - Outros nomes: **conexionismo**, **processamento distribuído em paralelo** e **computação neural**.
- Inspiradas na hipótese de que a atividade mental consiste na atividade eletroquímica em redes de células cerebrais chamadas **neurônios**.

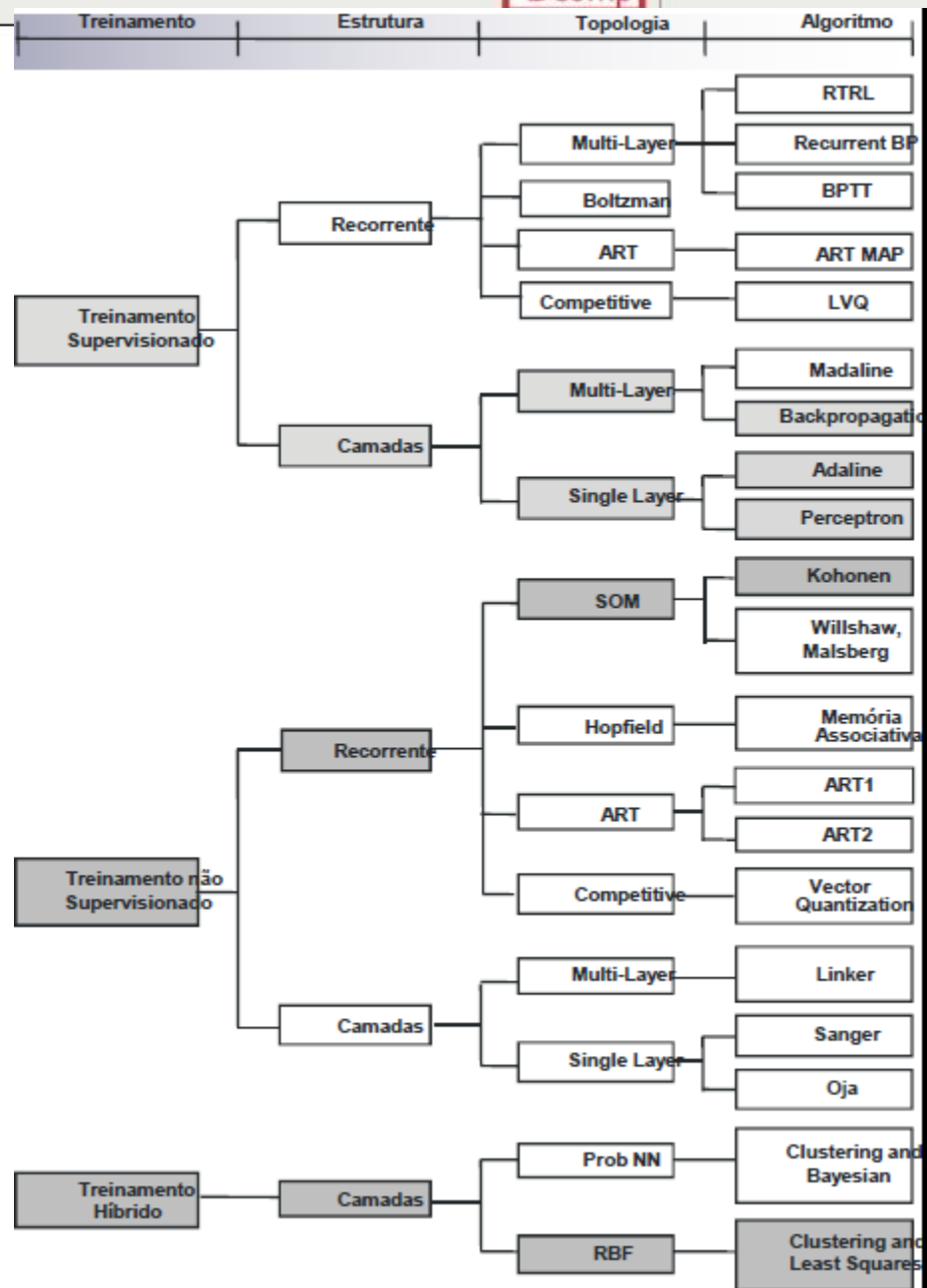
Introdução

- Uma RNA é um sistema composto por elementos de processamento relativamente simples, conectados entre si, formando uma rede.
- O desenvolvimento das RNAs foi motivado inicialmente pela busca de um modelo computacional do cérebro humano.



Redes Neurais

Existem diversos modelos computacionais que são tratados como redes neurais. Segundo o tipo de *treinamento*, pode ser aplicado numa RNA o treinamento *supervisionado*, ou *não supervisionado*. De acordo com a estrutura, as unidades de uma RNA podem organizadas em camadas, ou em estruturas recorrentes, quando parte (ou todas) as unidades estão conectadas entre si. Segundo a *topologia* da rede, ou seja, a forma com que as unidades estão conectadas, e o *algoritmo* de treinamento utilizado, pode-se então agrupar os mais diversos modelos de RNA encontrados na literatura.

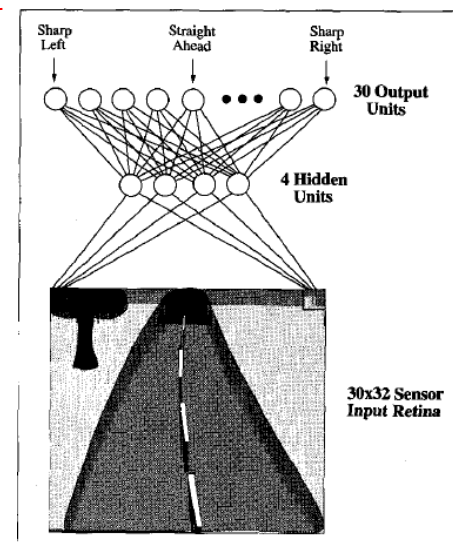
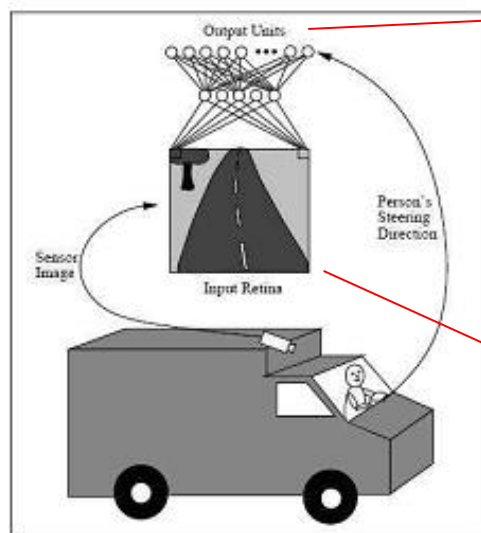


Introdução

- Métodos de aprendizado de RNAs fornecem uma abordagem robusta para aproximar funções com valores discretos e reais.
- RNAs são muito adequadas para problemas práticos, como:
 - reconhecimento de caracteres de manuscritos,
 - reconhecimento de palavras faladas e,
 - reconhecimento de faces.

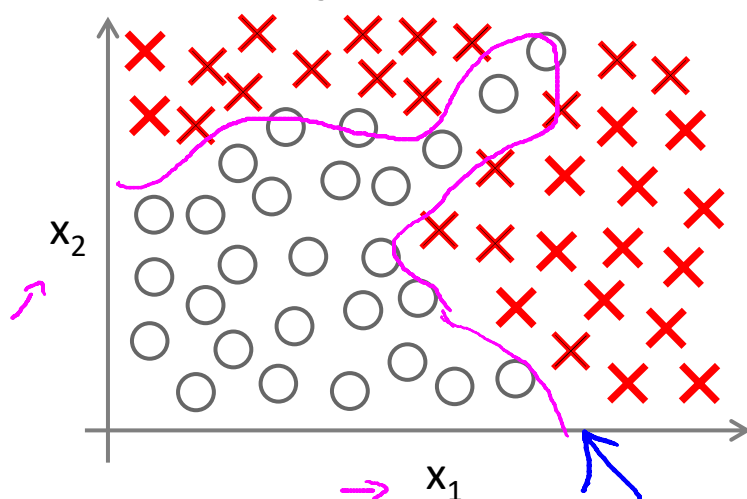
Aplicação de Redes Neurais

- O projeto ALVINN (*Autonomous Land Vehicle in a Neural Network*) (1989) foi um dos primeiros veículos robóticos inteligentes que utilizava redes neurais e processamento de imagem para conduzir um veículo de maneira autônoma em diversos tipos de ambiente.
- A rede neural, previamente treinada, fornecia como resultado os comandos que deveriam ser aplicados à direção do veículo.



Hipóteses Não-lineares

Classificação Não-linear



x_1 = size
 x_2 = # bedrooms
 x_3 = # floors
 x_4 = age
 $\dots$
 x_{100}

$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \theta_6 x_1 x_2^2 + \dots)$$

$$\rightarrow x_1^2, x_1 x_2, x_1 x_3, x_1 x_4 \dots x_1 x_{100} \\ x_2^2, x_2 x_3 \dots$$

$$\approx \underline{5000 \text{ feature}} \quad O(n^2)$$

$$\rightarrow x_1^2, x_2^2, x_3^2, \dots, x_{100}^2$$

$$\approx \frac{n^2}{2} = 10$$

$$\rightarrow x_1 x_2 x_3, x_1^2 x_2, x_{10} x_{11} x_{17}, \dots$$

$$O(n^3)$$

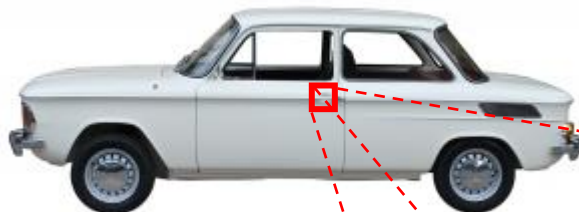
$$\underline{170,000}$$

$$h=100$$

Hipóteses Não-lineares

O que é isso?

Você vê assim:



Mas a câmera vê assim:

194	210	201	212	199	213	215	195	178	158	182	209
180	189	190	221	209	205	191	167	147	115	129	163
114	126	140	188	176	165	152	140	170	106	78	88
87	103	115	154	143	142	149	153	173	101	57	57
102	112	106	131	122	138	152	147	128	84	58	66
94	95	79	104	105	124	129	113	107	87	69	67
68	71	69	98	89	92	98	95	89	88	76	67
41	56	68	99	63	45	60	82	58	76	75	65
20	43	69	75	56	41	51	73	55	70	63	44
50	50	57	69	75	75	73	74	53	68	59	37
72	59	53	66	84	92	84	74	57	72	63	42
67	61	58	65	75	78	76	73	59	75	69	50

Hipóteses Não-lineares

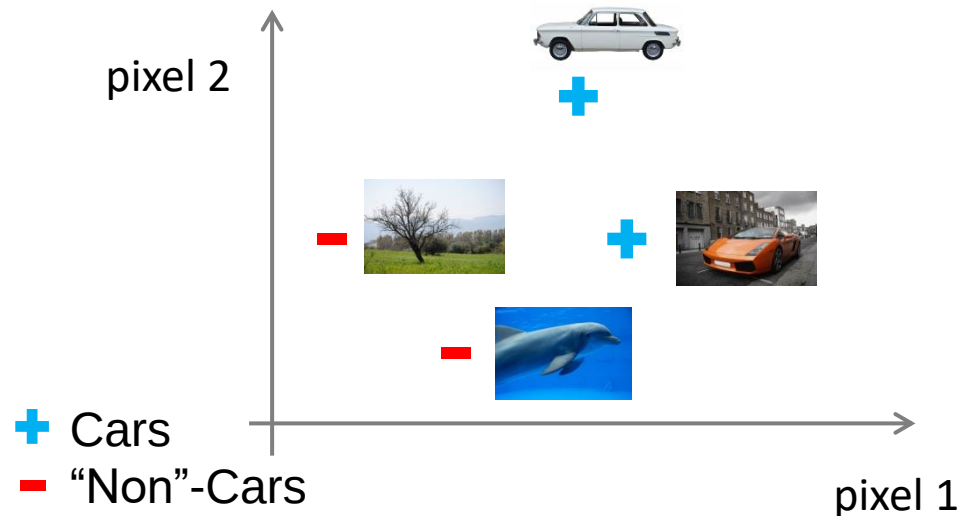
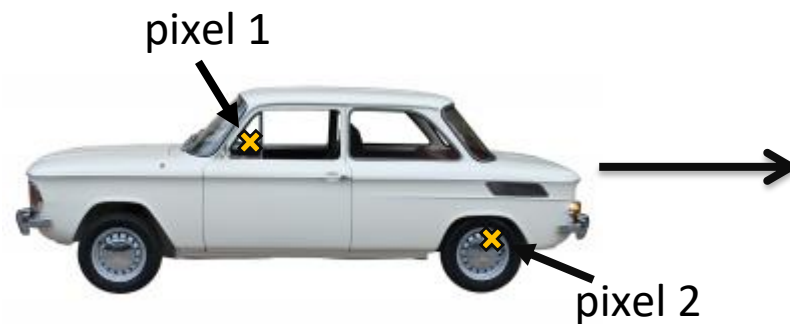
Visão Computacional: Detecção de Carros



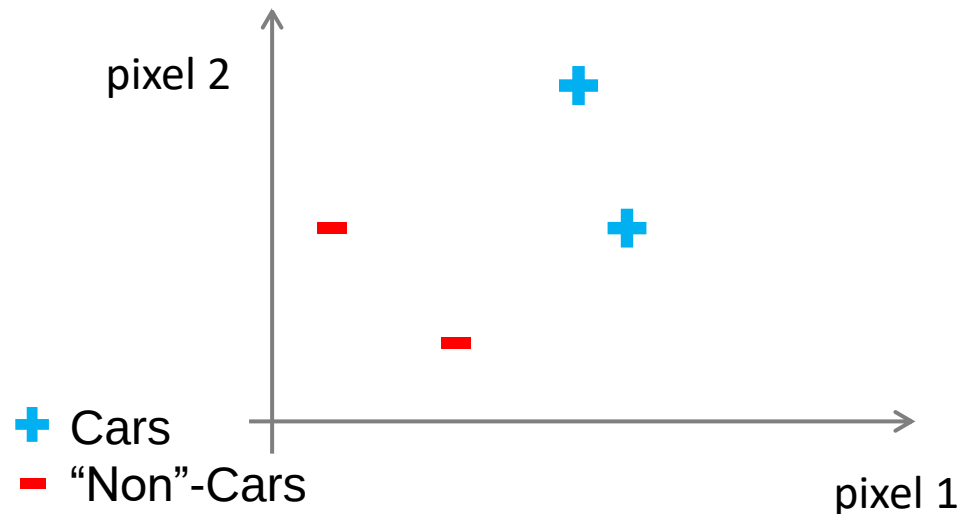
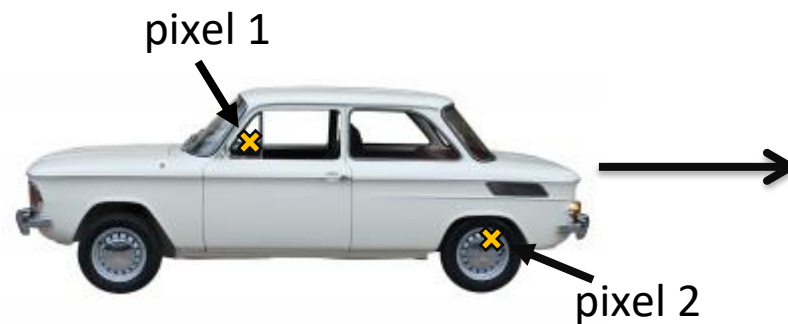
Teste:
O que é isso?



Hipóteses Não-lineares



Hipóteses Não-lineares



Hipóteses Não-lineares



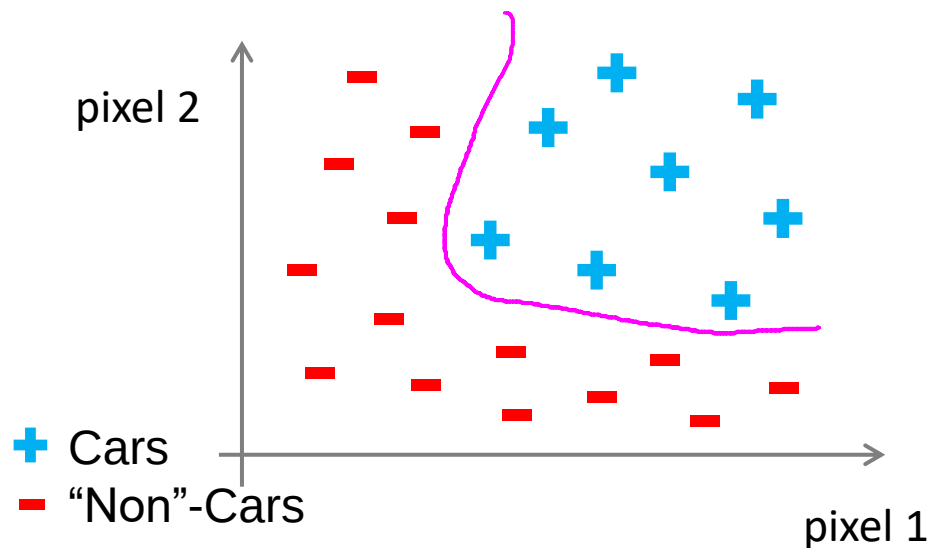
Algoritmo de
Aprendizado

50 x 50 pixel images → 2500
pixels

$n = 2500$

(7500 if RGB)

0-255



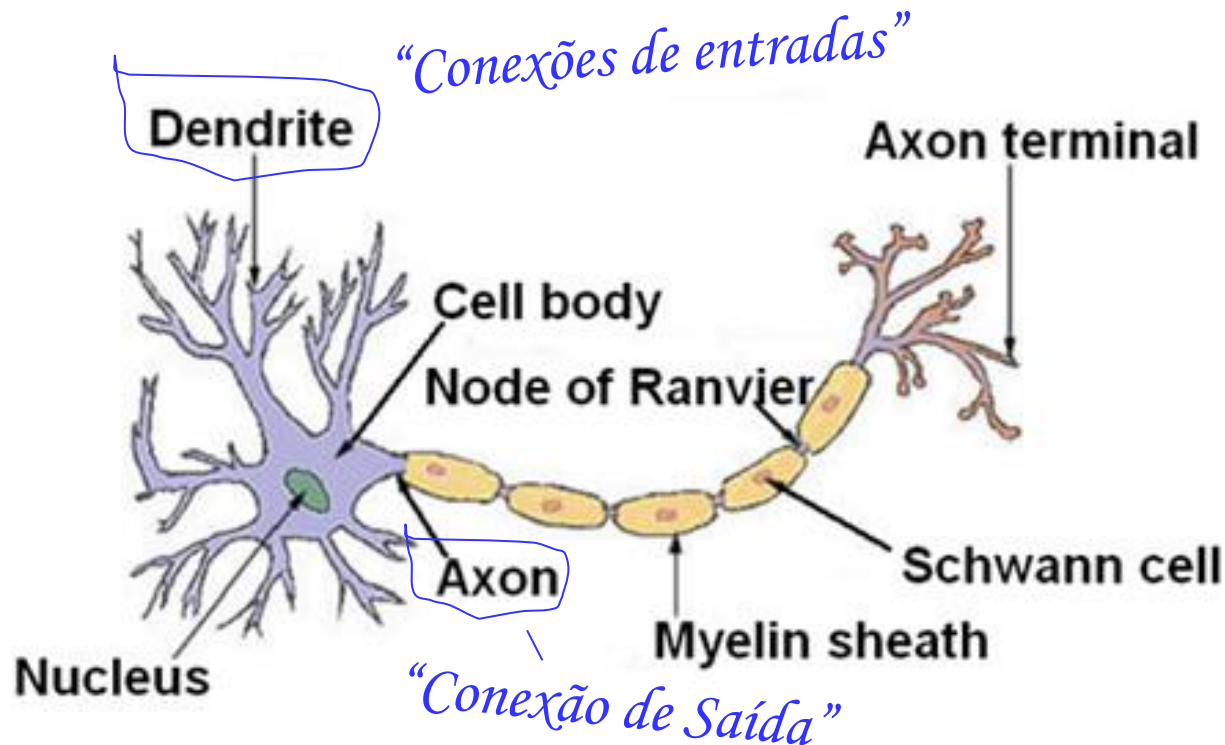
$$x = \begin{bmatrix} \text{pixel 1 intensity} \\ \text{pixel 2 intensity} \\ \vdots \\ \text{pixel 2500} \\ \text{intensity} \end{bmatrix}$$

Motivação Biológica

- O estudo de Redes Neurais Artificiais (RNA) foi inspirado em observações do sistema de aprendizado biológico.
 - Estes sistemas são construídos a partir de muitas redes complexas de neurônios interconectados.
- Uma RNA é um sistema composto por elementos de processamento relativamente simples conectados entre si, formando uma rede.
- O desenvolvimento das RNAS foi motivado inicialmente pela busca de um modelo computacional do cérebro humano.

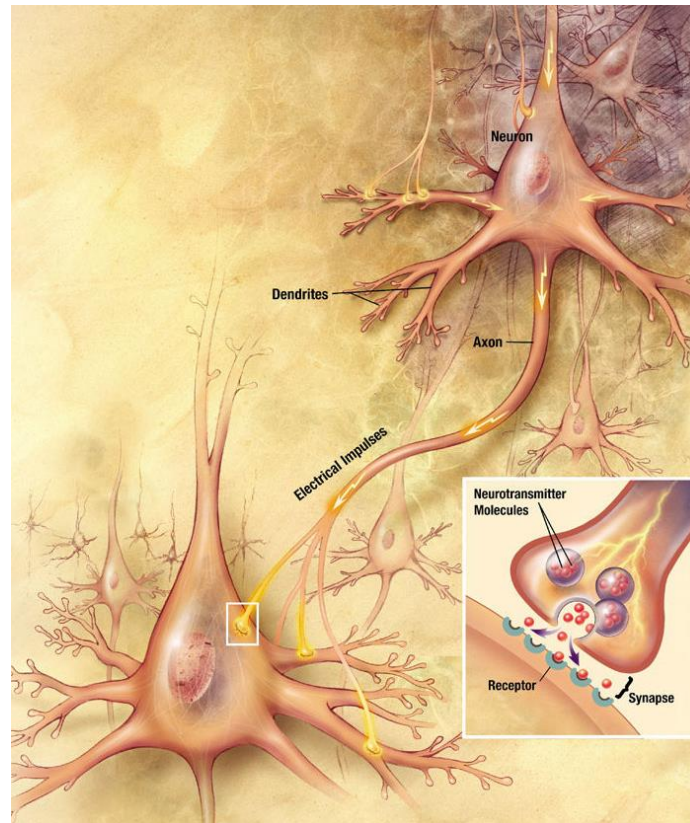
Motivação Biológica

Neurônio no cérebro



Motivação Biológica

Neurônio no cérebro



Motivação Biológica

- Algumas considerações:
 - Número de neurônios: $\approx 10^{10}$
 - Tempo de troca de neurônios: ≈ 0.001 segundo
 - Conexões por neurônio: $\approx 10^4$
 - Tempo de reconhecimento de cena: ≈ 0.1 segundo
 - 100 passos de inferência parece não ser suficiente
 - Especulação: muita computação paralela

Motivação Biológica

- Propriedades de Redes Neurais Artificiais (RNA):
 - Muitas unidades de troca como os neurônios;
 - Muitas interconexões ponderadas entre unidades;
 - Altamente paralelo, processo distribuído.

Motivação Biológica

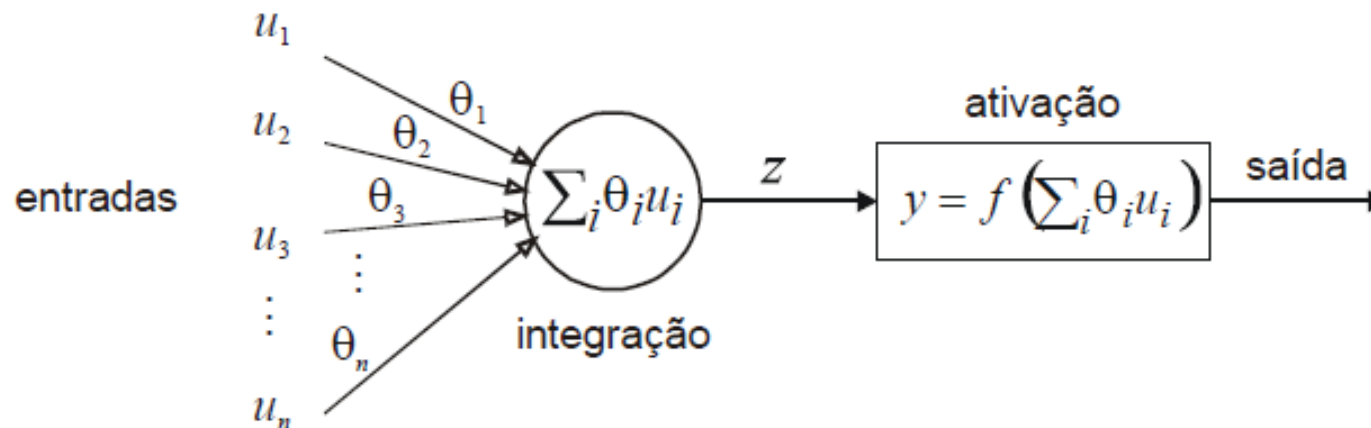
- Redes Neurais:
 - Origem: Algoritmos que tentam imitar o cérebro.
 - Foram amplamente usadas entre os anos 80 e 90, mas teve sua popularidade reduzida no final dos anos 90.
- Ressurgimentos recentes: técnica do estado-da-arte para muitas aplicações.

Modelo de Neurônio

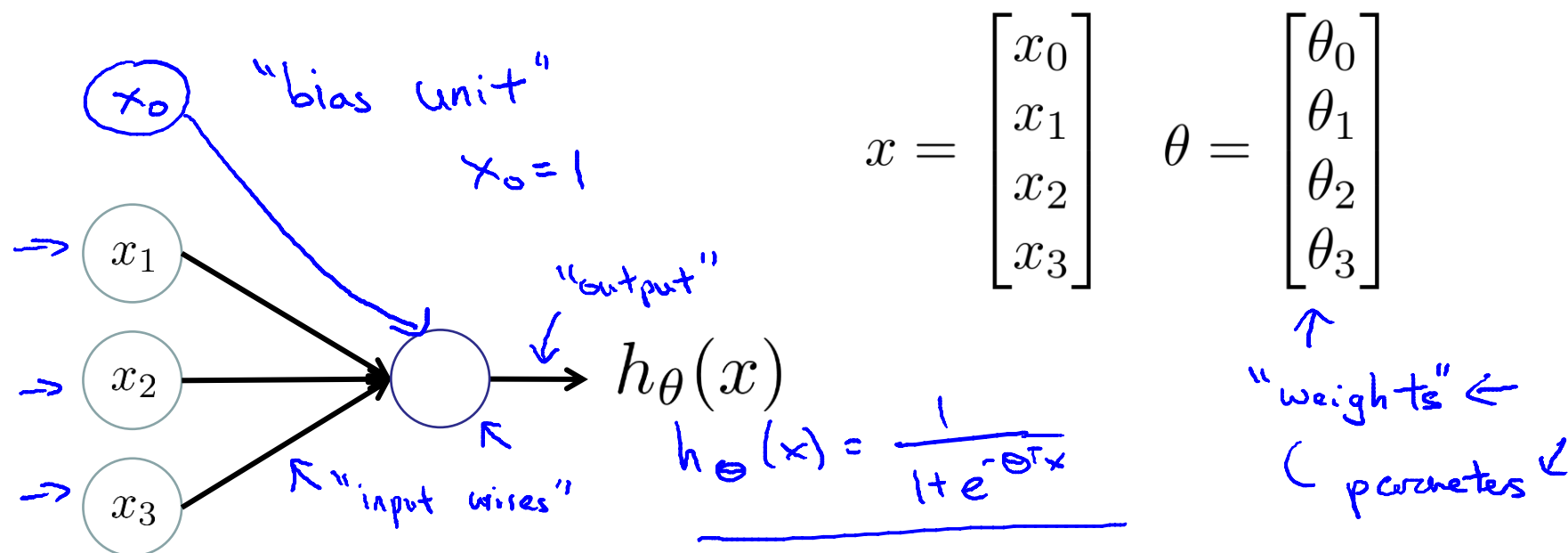
O neurônio de McCulloch-Pitt é um modelo matemático do neurônio biológico, onde a saída y é calculada a partir das entradas u_i como:

$$y = f(z) = f\left(\sum_i \theta_i u_i\right)$$

onde os parâmetros θ_i representam os pesos das conexões sinápticas; o valor z é chamado ativação e f é a função de ativação.



Modelo de Neurônio

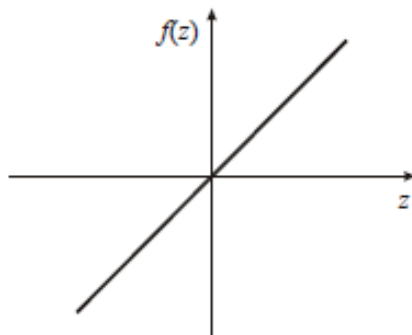


Função de ativação sigmóide (logística)

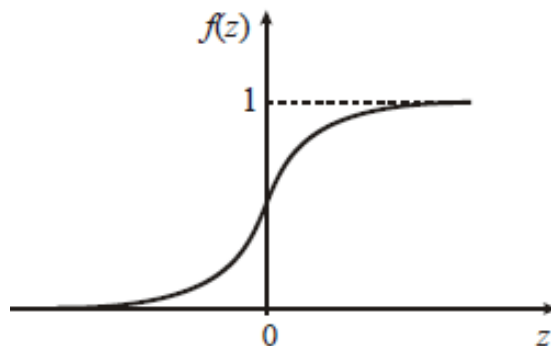
$$g(z) = \frac{1}{1 + e^{-z}}$$

Funções de Ativação

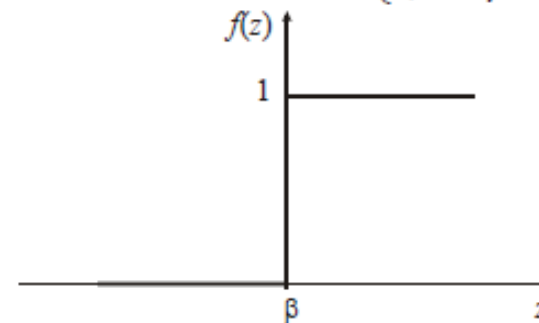
Linear: $y = f(z) = z = \sum_i \theta_i u_i$



Sigmóide: $y = f(z) = \frac{1}{1 + e^{-\alpha z}}$

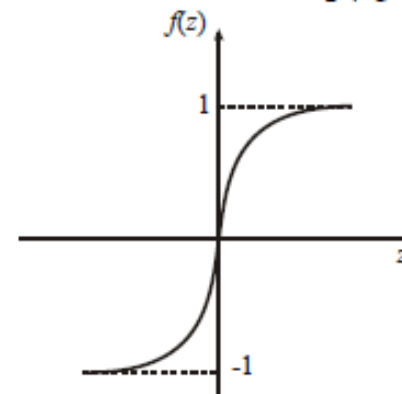


Lógica: $y = f(z) = \begin{cases} 0, & z < \beta \\ 1, & z \geq \beta \end{cases}$



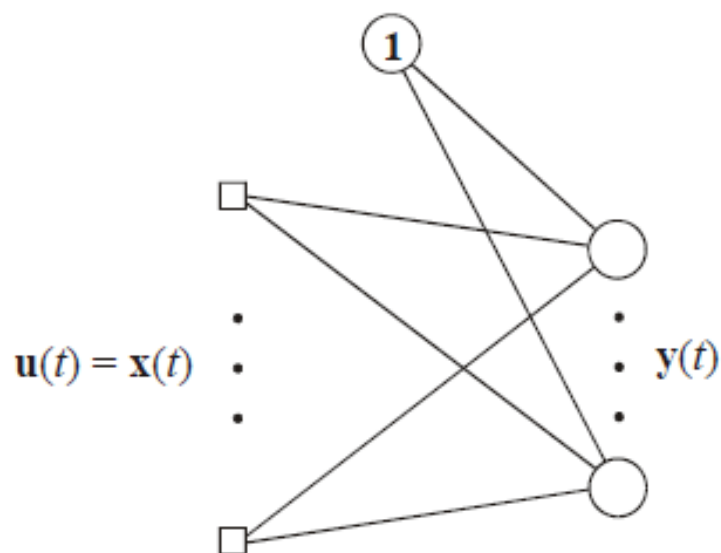
Tan. Hiperbólica:

$y = f(z) = \tanh(z) = \frac{1 - e^{-2z}}{1 + e^{-2z}}$



Modelos em Camadas Simples

A topologia de RNA mais simples que pode ser obtida com o modelo de neurônio McCulloch-Pitt é a topologia em camada simples, onde as entradas estão conectadas diretamente às unidades de saída.



Modelos em Camadas Simples

No modelo de RNA em camada simples, a saída da rede é calculada diretamente a partir da expressão do neurônio de *McCulloch-Pitt* como:

$$\mathbf{y}(t) = \mathbf{f}(\mathbf{z}(t)) = \mathbf{f}(\mathbf{u}(t) \cdot \Theta) = \mathbf{f}(\mathbf{x}(t) \cdot \Theta)$$

onde $\mathbf{x}(t) = (x_1(t), \dots, x_p(t))$ é a entrada e $\mathbf{y}(t) = (y_1(t), \dots, y_m(t))$ é a saída da rede.

Geralmente uma unidade *bias*, com valor constante e igual a 1, é adicionada a cada camada para ajustar o valor de transição de cada neurônio, de forma que o cálculo da saída é escrito como:

$$\mathbf{y}(t) = \mathbf{f}(\mathbf{z}(t)) = \mathbf{f}(\mathbf{u}(t) \cdot \Theta + \beta) = \mathbf{f}(\mathbf{x}(t) \cdot \Theta + \beta)$$

Modelos em Camadas Simples: o Perceptron

O Perceptron, um dos primeiros modelos de RNA desenvolvidos (1958), utiliza uma topologia em camada simples com função de ativação lógica bipolar, de forma que a saída é calculada como:

$$\mathbf{y}(t) = \mathbf{f}(\mathbf{x}(t) \cdot \Theta + \beta) = \text{sign}(\mathbf{x}(t) \cdot \Theta + \beta)$$

onde *sign* representa a função de ativação lógica bipolar:

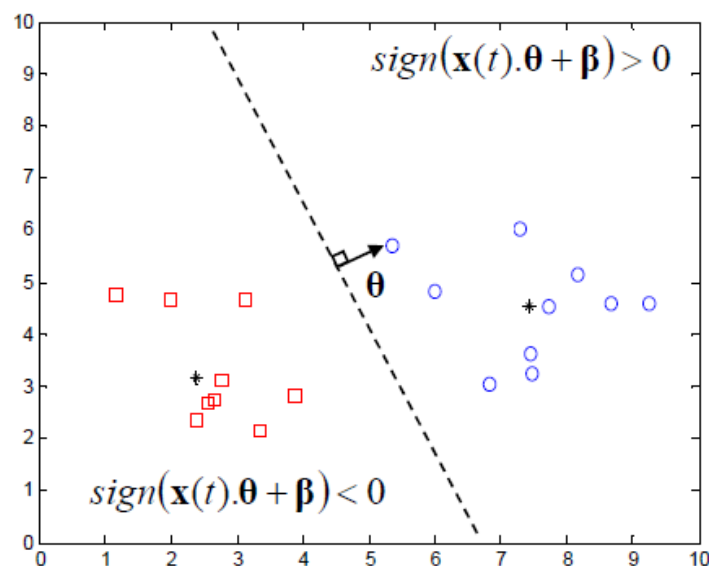
$$\text{sign}(\xi) = \begin{cases} +1, & \text{se } \xi > 0 \\ -1, & \text{se } \xi < 0 \end{cases}$$

O Perceptron é classificador, de forma cada unidade de saída é associada a uma classe, cujo reconhecimento é obtido pelo valor de ativação das unidades de saída:

$$y_i(t) = \begin{cases} +1, & \text{se } \mathbf{x}(t) \in \omega_i \\ -1, & \text{se } \mathbf{x}(t) \notin \omega_i \end{cases}$$

Modelos em Camadas Simples: o Perceptron

A importância do Perceptron, além de histórica, é que muitos elementos deste modelo e de seu algoritmo de ajuste de parâmetros servem de inspiração para modelos mais avançados. Entretanto, sua capacidade como classificador é limitada, uma vez que o modelo define um hiperplano de separação entre as classes, definido pelo vetor de parâmetros associado a cada classe.



Ajuste de Parâmetros: A regra do Perceptron

O algoritmo de ajuste do parâmetros do Perceptron é uma regra de *correção de erros*, que ajusta os pesos da rede de forma proporcional ao erro da saída. Desta forma conexões que produzem a saída correta não sofrem grande alteração. Este procedimento é semelhante ao que acontece com as conexões sinápticas dos neurônios naturais durante o processo de aprendizado.

Para o ajuste de parâmetros é necessário um conjunto de treinamento $T = \{(\mathbf{x}(t), \mathbf{v}(t)), t = 1..N\}$, onde, para cada exemplo t :

$\mathbf{x}(t) = (x_1(t), \dots, x_p(t))$ representa o vetor de atributos observados;

$\mathbf{v}(t) = (v_1(t), \dots, v_m(t))$ informa a classe correta associada ao exemplo:

$$v_i(t) = \begin{cases} +1, & \text{se } x(t) \in \omega_i \\ -1, & \text{se } x(t) \notin \omega_i \end{cases}$$

Ajuste de Parâmetros: A regra do Perceptron

O algoritmo de ajuste do parâmetros do Perceptron é sumarizado a seguir:

1. Inicialização dos pesos de conexões e taxa de treinamento (em geral $\eta = 1$)
2. Para $1 \leq t \leq N$ aplique a regra de ajuste dos parâmetros:
 - a. Calcular o erro da classe ω_i : $\varepsilon_i(t) = v_i(t) - y_i(t) = v_i(t) - \text{sign}(\mathbf{x}(t)\boldsymbol{\theta}_i(t) + \beta_i)$
 - b. Ajuste dos parâmetros:
$$\boldsymbol{\theta}_i(t+1) = \begin{cases} \boldsymbol{\theta}_i(t) + \eta \cdot \mathbf{x}(t)^T, & \text{se } v_i(t) \neq y_i(t) \\ \boldsymbol{\theta}_i(t), & \text{se } v_i(t) = y_i(t) \end{cases}$$
 - c. Se critério de parada não for satisfeito, retorne ao passo 2

3. FIM

Modelos em Camadas Simples: o ADALINE

O ADALINE, desenvolvido também no início da década de 1960, foi originalmente proposto para saída simples (duas classes), mas sua formulação pode ser facilmente estendida para problemas de múltiplas classes, normalmente conhecido como MADALINE, cuja formulação é:

$$y(t) = f(\mathbf{x}(t) \cdot \Theta + \beta) = \mathbf{x}(t) \cdot \Theta + \beta$$

O ADALINE utiliza função de ativação linear.

Ajuste de Parâmetros: A regra Delta

O ajuste de parâmetros do modelo é realizado pela minimização da norma quadrática do erro, calculada como:

$$J(t) = \frac{1}{2} \sum_{j=1..m} (v_j(t) - y_j(t))^2 = \frac{1}{2} [\mathbf{v}(t) - \mathbf{y}(t)][\mathbf{v}(t) - \mathbf{y}(t)]^T$$

A solução é obtida de forma iterativa ajustando-se os pesos das conexões como:

$$\boldsymbol{\theta}_i(t+1) = \boldsymbol{\theta}_i(t) + \Delta\boldsymbol{\theta}_i(t) = \boldsymbol{\theta}_i(t) - \eta \left. \frac{\partial J(t)}{\partial \boldsymbol{\theta}} \right|_t$$

Calculando-se as derivadas do critério de erro e substituindo na equação acima, chega-se a regra de ajuste de parâmetros do conhecida como regra delta, ou também regra de Widrow-Hoff:

$$\boldsymbol{\theta}_i(t+1) = \boldsymbol{\theta}_i(t) + \Delta\boldsymbol{\theta}_i(t) = \boldsymbol{\theta}_i(t) - \eta \cdot \delta_i(t) \cdot \mathbf{x}(t)^T$$

No caso em que a função de ativação é linear, $\delta_i(t) = (v_i(t) - y_i(t))$